Rheinische Friedrich-Wilhelms-Universität Bonn

Bachelorstudiengang Informatik

Prüfungsklausur

Semester: WS ☑ SS ☐ *2013/14*	Versuch: 1. ☑ 2. ☐
Modulbezeichnung: ***Systemnahe Programmierung***	
Modulnummer: *BA-INF 034/129*	Prüfungsdatum: 14. *Februar 2014*
1. Prüfer: *Prof. Dr. Michael Meier*	2. Prüfer: *Dr. Matthias Frank*

vom Prüfling auszufüllen:

Name:	Matrikel-Nr:
Vorname:	Semester:
Studiengang Bachelor Informatik ☐ Lehramt ☐ Informatik als Nebenfach ☐	

vom Prüfer auszufüllen:

Bewertung:

*Noten und Notenwerte: **sehr gut (1,0; 1,3) – gut (1,7; 2,0; 2,3) – befriedigend (2,7; 3,0; 3,3) – ausreichend (3,7; 4,0) – nicht ausreichend (5,0)**

Datum: 14. *Februar 2014*

---------- ----------

Unterschrift des 1. Prüfers Unterschrift des 2. Prüfers

Prof. Dr. Michael Meier
Dr. Matthias Frank

Bonn, den 14. Februar 2014

Hinweise zur Modulprüfung zu „Systemnahe Programmierung“ (BA-INF 034/129)

Wintersemester 2013/2014
9:30 Uhr – 11:00 Uhr

Hinweise (GENAU DURCHLESEN!)

- Neben Papier und Schreibutensilien sind **keine weiteren Hilfsmittel** erlaubt. Verwenden Sie nur dokumentenechte Stifte (z.B. keine Bleistifte). Verwenden Sie keine roten Stifte.
- Vergessen Sie nicht, Ihren Namen und die Matrikelnummer auf *jedes* Blatt zu schreiben. Blätter ohne diese Angaben werden nicht gewertet.
- Schreiben Sie Ihre Lösungen auf die Aufgabenblätter möglichst in die dafür vorgesehenen Felder. Sie können auch die Rückseiten verwenden. Weiteres Schreibpapier kann von den Betreuern angefordert werden. Benutzen Sie kein mitgebrachtes Papier.
- Bitte schreiben Sie in Ihrem eigenen Interesse deutlich. Für nicht lesbare Lösungen können wir keine Punkte vergeben.
- Klausurblätter dürfen nicht voneinander getrennt werden.
- Werden mehrere unterschiedliche Lösungen für eine Aufgabe abgegeben, so wird die Aufgabe nicht gewertet.
- Im Fall von Täuschungsversuchen wird die Klausur sofort mit 0 Punkten bewertet. Eine Vorwarnung erfolgt nicht.
- In der Klausur können Sie 90 Punkte erhalten. Achten Sie darauf, dass Sie im Schnitt nicht viel mehr als 1 Minute pro Punkt zur Verfügung haben. Mit 50% dieser Gesamtpunktzahl haben Sie die Klausur sicher bestanden.

Viel Erfolg!

1	**2**	**3**	**4**	**5**	**6**	**7**	**Gesamt**
12	14	15	20	15	12	2	90

Aufgabe 1 : (Multiple-Choice, 12 Punkte)

Bei den folgenden Multiple-Choice-Fragen wird für jedes richtig gesetzte Kreuz 1 Punkt vergeben. Bei falschen Antworten werden 0 Punkte vergeben.

	Aussage	Richtig	Falsch
1	Ein neu generierter Socket ist per default im non-blocking Mode, kann aber per fcntl() in den blocking Mode gesetzt werden.	☐	☐
2	Ports können nach einem sauberen Programmende nicht direkt wiederverwendet werden.	☐	☐
3	Ein TCP-Server bereitet sich durch die Abfolge von socket(), bind(), select() und accept() für die Annahme von eingehenden Verbindungs-aufbauwünschen vor.	☐	☐
4	Beim sog. „select loop“ für einen TCP-Server bedient ein einziger Prozess in einer endlos laufenden Schleife mit select() mehrere Sockets für alle aktiven Verbindungen. Eingehende Verbindungsaufbau-wünsche müssen separat überwacht werden.	☐	☐
5	Bei einem iterativen TCP-Server ist nur die Anzahl der aktiven Verbindungen begrenzt, nicht die der wartenden.	☐	☐
6	In der „Little-Endian-Byte-Order“ wird das niederwertigste Byte einer aus mehreren Bytes bestehenden Integer-Zahl an der niedrigsten Adresse im Speicher abgelegt.	☐	☐
7	Im Real-Mode gibt es einen automatischen Speichersegmentschutz.	☐	☐
8	Ein Zombi-Prozess ist ein Kindprozess, der noch läuft obwohl sein Vater beendet worden ist.	☐	☐
9	Threads laufen in getrennten Datenadressbereichen, teilen sich aber denselben Stack.	☐	☐
10	Bei einem Deadlock wartet ein Thread auf die Beendigung eines anderen.	☐	☐
11	Wenn ein Linux-Prozess mit geöffnetem File fork() aufruft, teilen sich der neue und der alte Prozess denselben File Offset.	☐	☐
12	x86-Rechner sind Little-Endian.	☐	☐

Sollten Sie der Meinung sein, dass eine (oder mehrere) der obigen Fragen missverständlich mit „Richtig“ oder „Falsch“ zu beurteilen sind, dann können Sie unten Ihre gewählte Antwort **kurz** begründen. Bitte achten Sie dabei selbst auf Ihre Zeiteinteilung: Es macht keinen Sinn, viele (oder gar alle) der Multiple-Choice-Fragen durch ausführlichen Text zu begründen. Es gibt **nur** für die **oben** gewählten Antworten Punkte, nicht für Ihre Erläuterungen.

Nr.	Ihre Erläuterung

Aufgabe 2: (I/O-Multiplexing in C, 14 Punkte [6 x 1 + (4 x 1 + 2 x 2)])

In der Vorlesung haben Sie die Hilfsfunktion `select()` kennen gelernt. `select()` stellt universelle Funktionalität für I/O-Multiplexing mehrerer Eingabequellen zur Verfügung. `select()` läßt sich sowohl als blockierender als auch nicht-blockierender Aufruf realisieren.

Informationen für Select:

```
struct timeval {
  long tv_sec  /*seconds*/
  long tv_usec /*microseconds*/
};

int select(int n, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout)
```

a) (6 Punkte) Erläutern Sie kurz die Parameter und den Rückgabewert von `select()`:

```
int n
```

```
fd_set *readfds
```

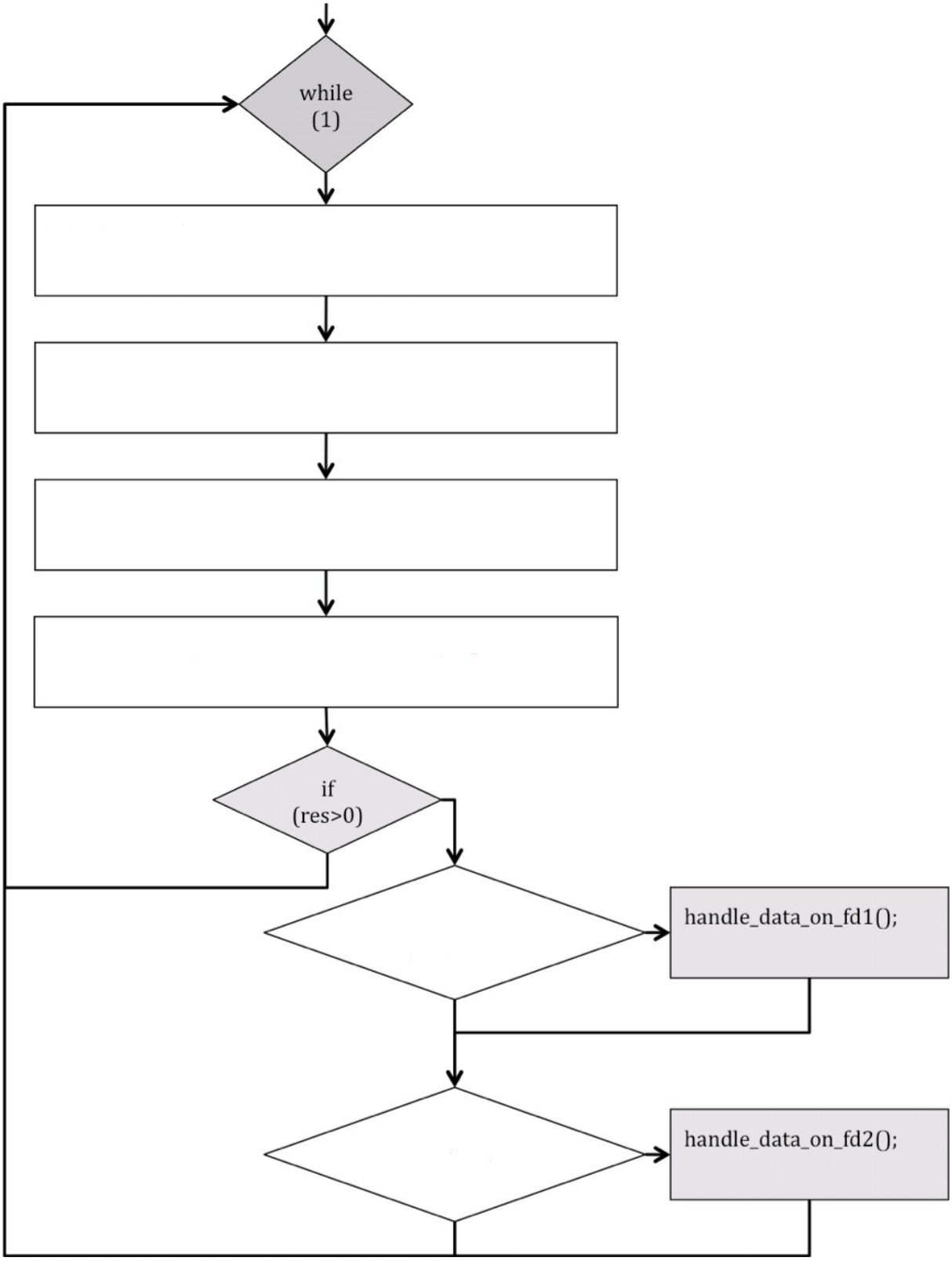
```
fd_set *writefds
```

```
fd_set *exceptfds
```

```
struct timeval *timeout
```

```
int select() – Rückgabewert:
```

b) (6 Punkte) Im Folgenden ist ein Ablaufschema für select mit zwei File-Descriptoren, fd1 und fd2, sowie die Set fd_set gegeben. Auf fd1 und fd2 soll lesend zugegriffen werden und der select -Aufruf soll blockierend sein. Füllen Sie die Kästen mit validem C-Code, so dass select zum I/O-Multiplexing zwischen den beiden File Descriptoren genutzt wird.



Aufgabe 3: (Assembler, 15 Punkte [3+6+2+4])

Gegeben ist das im Folgenden dargestellte Assembler-Programm für einen 32-Bit x86-Prozessor. Es besteht aus einer Hauptroutine und einer Unterroutine `getMax` mit der C-Signatur `int getMax(int *array, unsigned int size).`

Aus Gründen der Übersichtlichkeit ist das Programm auf diese und die folgenden Seiten verteilt.

Hauptroutine des Assemblerprogramms:

```
.data

     # Ein int-Array mit arraysize Elementen
array:
     .long 42, 4711, 23, 666, 34
     arraysize = 5

.text

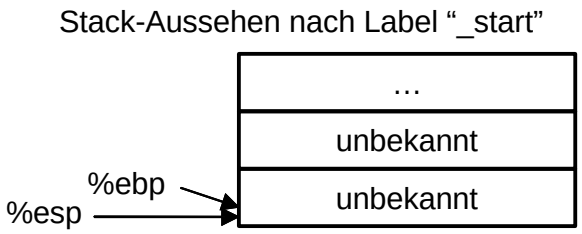
.global _start

_start:

     # call getMax(&array, arraysize)
     pushl     $arraysize
     pushl     $array
     call      getMax
     # Parameter wieder vom Stack nehmen
     addl      $8, %esp

     # Prozess beenden
     movl      $1, %eax
     movl      $0, %ebx
     int       $0x80
```

Beim Eintritt in die Hauptroutine des Assemblerprogramms hat der Stack das folgende Aussehen (Zeitpunkt vor der Ausführung des ersten Befehls nach dem Label „_start“):



a) (3 Punkte) Ergänzen Sie nun den Stack (Belegung der Zellen mit Konstanten bzw. kurze Erklärung des Inhaltes), wie er nach dem Betreten der Unterroutine „`getMax`“ (siehe nächste Seite) beim Erreichen des Labels „`Zeitpunkt_1`“ aussieht. Kennzeichnen Sie auch die dann aktuelle Position des Basepointers `%ebp` und des Stackpointers `%esp`.

Platz für Ihre Lösung: **Reserve-Platz für Ihre Lösung:**

(bitte streichen Sie eine ungültige Lösung deutlich)

…
unbekannt
unbekannt

...
unbekannt
unbekannt

b) (6 Punkte) Die Unterroutine `int getMax(int *array, unsigned int size)` erwartet als Parameter einen Zeiger auf ein Array von Integer-Werten, mit `size` vielen Elementen. Sie soll das Maximum aller Array-Elemente zurückliefern und der in der Vorlesung besprochenen „cdecl Calling Convention“ gehorchen.
In das Programm haben sich **sechs** fehlerhafte Zeilen eingeschlichen. Finden Sie die sechs Zeilen, und korrigieren Sie die Fehler in den freien Feldern neben den Zeilen.

```
# int getMax(int *array, unsigned int size)
getMax:
    # Stack frame
    pushl    %ebp                  _____________________________
    movl     %esp, %ebp            _____________________________
Zeitpunkt_1:
    # 1 lokale Integer-Variable
    subl     $1, %esp              _____________________________
    # Rette Register (wg. cdecl)
    pushl    %ecx                  _____________________________
    movl     $0, -4(%ebp)          _____________________________
    # Hole Zeiger auf Array und Größe
    movl     12(%ebp), %esi        _____________________________
    movl     8(%ebp), %ecx         _____________________________

arrayloop:
    movl     (%esi), %eax          _____________________________
    cmpl     %eax, -4(%ebp)        _____________________________
    jnl      nichtkleiner          _____________________________
    movl     %eax, -4(%ebp)        _____________________________
nichtkleiner:
    addl     $4, (%esi)            _____________________________
    loop     arrayloop             _____________________________

    movl     -4(%ebp), %eax        _____________________________
    # Register wiederherstellen
    popl     %ecx                  _____________________________
    leave                          _____________________________
    ret                            _____________________________
```

c) (2 Punkte) Ersetzen Sie den Befehl „leave“ in den folgenden Zeilen durch eine äquivalente Folge von Assemblerbefehlen, wobei eine beliebige Auswahl aus den folgenden Befehlen erlaubt ist: {movl, pushl, popl, addl, subl, incl, decl}

Lösung:

_____

_____

_____

_____

d) (4 Punkte) In einem typischen Assemblerprogramm tauchen verschiedene Arten der Adressierung auf. Wie nennt man die folgenden Adressierungsarten?

`movl $1, %eax`	
`movl (1000), %eax`	
`movl (%ecx), %eax`	
`movl 8(%ebp), %eax`	

Aufgabe 4: (Thread-Synchronisation, insbes. Mutex, Semaphor, 20 Punkte + ggf. 2 Bonuspunkte [5 + 5 + 4 + 2 Bonus + 6])

a) (5 Punkte) In der Vorlesung wurden die folgenden fünf Konzepte für Thread-Synchronisation im Detail besprochen. Nennen Sie kurz und prägnant den Einsatzzweck bzw. charakteristische Eigenschaften des jeweiligen Konzeptes:

1. Mutex Variablen

_____

_____

2. Condition Variablen

_____

_____

3. Semaphore

_____

_____

4. Barriers

_____

_____

5. Read-write Locks

_____

_____

Betrachten Sie bei den weiteren Aufgabenteilen b) bis e) die folgenden gegebenen Funktionen, die allein mit Hilfe von Mutex-Variablen Semaphore realisieren - konkret die drei wesentlichen Funktionen für Semaphore, nämlich SEM_init(), SEM_wait() und SEM_post().
Auf eine Identifikation des Semaphors wird aus Gründen der besseren Lesbarkeit verzichtet, d.h. in einem Thread-fähigen Programm könnte nur eine Instanz unseres Semaphors benutzt werden.

```
int Sem_counter ;  // globale Counter-Variable

void SEM_init(int counter_init) {
     mutex_init(&CounterMutex, NULL)
     mutex_init(&SemMutex, NULL)

     mutex_lock(&CounterMutex);
     Sem_counter = counter_init;
     mutex_unlock(&CounterMutex);
}

void SEM_wait() {
     int block = 0;

     mutex_lock(&CounterMutex);
     Sem_counter--;
     printf("counter-- = %d\n", Sem_counter);
     If (Sem_counter <=0) block = 1;
     mutex_unlock(&CounterMutex);

     If (block) {
          printf("Vor lock = %d\n", Sem_counter);
          mutex_lock(&SemMutex);
          printf("Nach lock = %d\n", Sem_counter);
     }
}

void SEM_post() {
     mutex_lock(&CounterMutex);
     Sem_counter++;
     mutex_unlock(&CounterMutex);

     printf("post! = %d\n", Sem_counter);
     mutex_unlock(&SemMutex);
          // unlock eines nicht gelockten Mutex
          // bewirkt keinen Schaden
}
```

Die hier gegebene Implementation von SEM_wait() realisiert das Prinzip von wait() mit negativem Zähler:

1) Dekrementiere den Zähler um 1
2) Wenn Zähler < 0, dann blockiere
3) Nutze die Ressource (hier: verlasse SEM_wait())

b) (5 Punkte) Skizzieren Sie die Ausgabe (siehe printf()-Statements in SEM_wait() und SEM_post()) für folgenden zeitlichen Ablauf von Semaphor-Operationen. Nehmen Sie dabei an, dass die Funktionen SEM_wait() und SEM_post() nicht aufgrund von Thread-Scheduling unterbrochen werden (Ausnahme: Blockieren des Mutex <SemMutex> ist möglich).

SEM_init(3)

SEM_wait()

SEM_wait()

SEM_wait()

SEM_wait()

SEM_wait()

SEM_post()

Schreiben Sie Ihre Lösung neben die Aufrufe oben oder hier unterhalb:

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

c) (4 Punkte) Die in der Aufgabe gegebene Implementation von SEM_wait() realisiert das Prinzip von wait() mit negativem Zähler:

1) Dekrementiere den Zähler um 1
2) Wenn Zähler < 0, dann blockiere
3) Nutze die Ressource (hier: verlasse SEM_wait())

Der Vergleich auf den Zählerstand in SEM_wait() lautet jedoch:

```
If (Sem_counter <=0) block = 1;
```

Erläutern Sie, warum ein Vergleich auf „< 0“ nicht das gewünschte Ergebnis bringen würde! (Hinweis: bedenken Sie die printf()-Ausgaben aus Aufgabe b) auf dem Weg zu Ihrer Antwort)

d) **Zusatzfrage!!!** Durch beantworten dieser Frage können Sie bei dieser Aufgabe **2 zusätzliche Punkte** bekommen:

In welcher Hinsicht könnte die oben genannte Implementierung eines Semaphors durch Mutex-Variablen im realen Einsatz problematisch sein? Denken Sie an die quasi-parallele Ausführung von Threads und die mögliche Unterbrechung durch Thread-Scheduling.

e) (6 Punkte) Die gegebene Implementierung von SEM_wait() soll nun so verändert werden, dass das folgende Prinzip mit nicht-negativem Zähler für Semaphore realisiert wird:

1) Wenn Zähler = 0, dann blockiere
2) Dekrementiere den Zähler um 1
3) Nutze die Ressource (hier: verlasse SEM_wait())

Schreiben Sie SEM_wait() nun entsprechend um. printf()-Ausgaben wie im gegebenen Beispiel brauchen dabei **nicht** enthalten sein. SEM_init() und SEM_post() brauchen nicht verändert werden (und sind zur Übersichtlichkeit nochmals dargestellt).

Denken Sie an korrekte Realisierung des Vergleiches des Zählerstandes (vgl. Aufgabenteil c).

```
int Sem_counter ;  // globale Counter-Variable

void SEM_init(int counter_init) {
     mutex_init(&CounterMutex, NULL)
     mutex_init(&SemMutex, NULL)

     mutex_lock(&CounterMutex);
     Sem_counter = counter_init;
     mutex_unlock(&CounterMutex);
}

void SEM_wait() {
```

Platz für Ihre Lösung:

```
} // Ende von SEM_wait()

void SEM_post() {
     mutex_lock(&CounterMutex);
     Sem_counter++;
     mutex_unlock(&CounterMutex);

     mutex_unlock(&SemMutex);
          // unlock eines nicht gelockten Mutex
          // bewirkt keinen Schaden
}
```

Aufgabe 5: (PThreads und Parameterübergabe, 15 Punkte [3+2+10])

a) (3 Punkte) Nennen und beschreiben Sie jeweils in einem Satz alle in der Vorlesung besprochenen Arten, wie ein PThread beendet werden kann!

Lösung:

b) (2 Punkte) Nennen Sie kurz die wesentlichen Unterschiede zwischen den beiden Threadzuständen *Joinable* und *Detached*!

Lösung:

c) (10 Punkte) Gegeben sei folgendes Programmfragment (siehe nächste Seite), bei dem die Funktion *main* eine Unterroutine *createThread(int a, int b)* aufruft, welche einen neuen Thread starten und diesem die Werte *a* und *b* übergeben soll. Der Thread, welcher aus der Funktion *workThreadFunction* bestehen soll, soll nichts anderes tun, als die Summe der beiden Werte *a+b* zu berechnen und zurückzugeben. Die *main*-Funktion soll das Ergebnis des Threads ausgeben.

Ergänzen Sie die Signatur der Funktion *workThreadFunction.* Implementieren Sie die Funktionen *workThreadFunction* und *createThread* mit der oben beschriebenen Funktionalität. Ergänzen Sie die Funktion *main* so, dass der Rückgabewert von *workThreadFunction* ausgegeben wird.

Sie können davon ausgehen, dass der neue Thread automatisch im Zustand *Joinable* startet - Sie brauchen keine Threadattribute setzen. Ebenso brauchen Sie sich nicht um eventuelle Fehlerbehandlungen oder Speicherfreigaben zu kümmern. Sie dürfen den vorhandenen Code **nicht** verändern oder außerhalb der von Ihnen zu ergänzenden Funktionen erweitern.

Lösung:

```
pthread_t workThreadID;
struct wtParams {
     int param1, param2;
};


_______ workThreadFunction(_____________________) {









}


void createThread(int a, int b) {









}


void main(int argc, char **argv) {
     int threadResult;
     createThread(3, 5);









     printf("Resultat: %d\n", threadResult);
}
```

Reserve-Platz für Ihre Lösung:

(Bitte streichen Sie eine ungültige Lösung deutlich)

```
pthread_t workThreadID;
struct wtParams {
     int param1, param2;
};


_______ workThreadFunction(_____________________) {









}


void createThread(int a, int b) {









}


void main(int argc, char **argv) {
     int threadResult;
     createThread(3, 5);









     printf("Resultat: %d\n", threadResult);
}
```

Aufgabe 6: (C Socket-Programmierung, 12 Punkte [6 x 2])

Aus der Vorlesung ist Ihnen die folgende TCP Client-Server-Anwendung bekannt. Der Client erhält den Host-Namen des Servers als Kommandozeilenparameter und liest einen String von der Standardeingabe ein. Dieser soll an den Server geschickt werden, der den String in Großbuchstaben umwandeln und an den Client zurücksenden soll. Der Client gibt den empfangenen String auf der Standardausgabe aus.
In das Programm haben sich an den 6 markierten Stellen Fehler eingeschlichen. Schreiben Sie jeweils in den Kasten „Korrektur“ eine korrigierte Version der entsprechenden Zeile. Kennzeichnen Sie das Löschen einer Zeile durch das Wort „Löschen“ an der entsprechenden Stelle im Korrekturkasten.

Beachten Sie, dass Client und Server zusammenarbeiten sollen!

```
#include <stdio.h>
#include <unistd.h>
#include <ctype.h>
#include <string.h>
#include <netinet/in.h>
#include <netdb.h>
#define MAXLINE 4096

int main(int argc, char **argv) {                // TCP Server
  struct sockaddr_in myaddr, remaddr;
  socklen_t addrlen;
  int li, newfd, nbytes,i;
  char buf[MAXLINE];

  bzero(&myaddr, sizeof(myaddr));
  myaddr.sin_family = AF_INET;
  myaddr.sin_addr.s_addr = INADDR_ANY;
  myaddr.sin_port = htons(4711);
  addrlen = sizeof(struct sockaddr_in);

  if ((li = socket(AF_INET, SOCK_STREAM, 0)) < 0) return 1;
  if (bind(li, (struct sockaddr *)&myaddr, sizeof(myaddr)) < 0)
    return 1;
  printf("Local Port: %d\n", myaddr.sin_port);                  (1)
  if (listen(li, 10) == -1) return 1;
  while (1) {
    if ((li = accept(newfd, (struct sockaddr *)&remaddr,        (2)
                          &addrlen)) >= 0) {                    (2)
      while((nbytes = read(newfd, buf, sizeof(buf) - 1)) > 0) {
        for( i = 0; i<nbytes; i++)
          buf[ i ] = toupper( buf[ i ] );
        write(newfd, buf, nbytes, 0,                            (3)
                (struct sockaddr *)&remaddr, &addrlen));        (3)
      }
      close(newfd);
    }
  }
  return 0;
}
```

Korrektur – TCP Server:

(1)

(2)

(3)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netdb.h>
#define SERVPORT 4711
#define MAXLINE 4096

int main(int argc, char **argv) {        // TCP Client
  int fd;
  struct sockaddr_in serv_addr;
  struct hostent *dns_info;
  char tempstr[MAXLINE] = "";

  if (argc != 2) {
    printf("usage: %s server-hostname \n", argv[0]);
    return (1);
  }
  dns_info = gethostbyname(argv[1]);
  if (!dns_info) {
    printf("Keine IP-Adresse zugeordnet!\n");
    return (1);
  }
  bzero(&serv_addr, sizeof(serv_addr));
  serv_addr.sin_family = AF_ROUTE;                              (4)
  serv_addr.sin_port = htons(SERVPORT);
  serv_addr.sin_addr = *(struct in_addr *)dns_info->h_addr;

  if ((fd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) return 1;      (5)
  if ((listen(fd, (struct sockaddr *)&serv_addr,                (6)
                   sizeof(struct sockaddr_in))) < 0 ) return 1; (6)

  printf("Sende Text: ");
  while ((int)tempstr[0] != 10) { // Teste auf Line Feed
    fgets(tempstr, MAXLINE, stdin);
    write(fd, tempstr, strlen(tempstr));
    read(fd, tempstr, MAXLINE);
    printf("Empfange Text: %s\n\n", tempstr);
    printf("Sende Text: ");
  }
  close(fd);
  return 0;
}
```

Korrektur – TCP Client:

(4)

(5)

(6)

Aufgabe 7: (Sockets, 2 Punkte [1+1])

a) Ordnen Sie die Eigenschaften *verbindungsorientiert, verbindungslos, zuverlässig, unzuverlässig* den entsprechenden Socket-Typen zu.

SOCK_STREAM	**SOCK_DGRAM**

b) Welches Transportprotokoll verwenden die jeweiligen Socket-Typen bei Verwendung mit der AF_INET-Adressfamilie?

i) SOCK_STREAM:

ii) SOCK_DGRAM:

Signaturen relevanter Bibliotheksfunktionen

int **pthread_create**(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void*), void *arg);
int **pthread_attr_init**(pthread_attr_t *attr);
int **pthread_attr_destroy**(pthread_attr_t *attr);
void **pthread_exit**(void *value_ptr);
int **pthread_join**(pthread_t thread, void **value_ptr);
int **pthread_detach**(pthread_t thread);
int **pthread_cancel**(pthread_t thread);
int **pthread_setcancelstate**(int state, int *oldstate);
int **pthread_setcanceltype**(int type, int *oldtype);
void **pthread_testcancel**(void);
int **pthread_mutex_init**(pthread_mutex_t *mutex, pthread_mutexattr_t *attr);
int **pthread_mutex_destroy**(pthread_mutex_t *mutex);
int **pthread_mutexattr_init**(pthread_mutexattr_t *attr);
int **pthread_mutexattr_destroy**(pthread_mutexattr_t *attr);
int **pthread_mutexattr_settype**(pthread_mutexattr_t *attr, int type);
int **pthread_mutexattr_gettype**(const pthread_mutexattr_t *attr, int *type);
int **pthread_mutex_lock**(pthread_mutex_t *mutex);
int **pthread_mutex_trylock**(pthread_mutex_t *mutex);
int **pthread_mutex_unlock**(pthread_mutex_t *mutex);
int **pthread_cond_init**(pthread_cond_t *cond, const pthread_condattr_t *attr);
int **pthread_cond_destroy**(pthread_cond_t *cond);
int **pthread_condattr_init**(pthread_condattr_t *attr);
int **pthread_condattr_destroy**(pthread_condattr_t *attr);
int **pthread_cond_wait**(pthread_cond_t *cond, pthread_mutex_t *mutex);
int **pthread_cond_timedwait**(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);
int **pthread_cond_signal**(pthread_cond_t *cond);
int **pthread_cond_broadcast**(pthread_cond_t *cond);
int **sem_init**(sem_t *sem, int pshared, unsigned int value);
int **sem_destroy**(sem_t *sem);
int **sem_wait**(sem_t *sem);
int **sem_trywait**(sem_t *sem);
int **sem_post**(sem_t *sem);
sem_t ***sem_open**(const char *name, int oflag);
sem_t ***sem_open**(const char *name, int oflag, mode_t mode, unsigned int value);
int **sem_close**(sem_t *sem);
int **sem_unlink**(const char *name);
int **pthread_barrier_init**(pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned count);
int **pthread_barrier_destroy**(pthread_barrier_t *barrier);
int **pthread_barrier_wait**(pthread_barrier_t *);
int **pthread_rwlock_init**(pthread_rwlock_t *rwlock, const pthread_rwlockattr_t *attr);
int **pthread_rwlock_destroy**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_rdlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_tryrdlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_trywrlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_unlock**(pthread_rwlock_t *rwlock);
pid_t **getpid**(void);
pid_t **getppid**(void);

int **system**(const char *command);
pid_t **fork**(void);
int **execl**(const char *path, const char *arg0, ... /*, (char *)0 */);
int **execv**(const char *path, char *const argv[]);
int **execle**(const char *path, const char *arg0, ... /*, (char *)0, char *const envp[]*/);
int **execve**(const char *path, char *const argv[], char *const envp[]);
int **execlp**(const char *file, const char *arg0, ... /*, (char *)0 */);
int **execvp**(const char *file, char *const argv[]);
void **exit**(int status);
void **abort**(void);
pid_t **wait**(int *stat_loc);
pid_t **waitpid**(pid_t pid, int *stat_loc, int options);
int **pipe**(int fd[2]);
int **dup**(int fd);
int **dup2**(int oldfd, int newfd);
FILE ***popen**(const char *cmdstring, const char *type);
int **pclose**(FILE *fp);
int **mkfifo**(const char *pathname, mode_t mode);
mqd_t **mq_open**(const char *name, int oflag);
mqd_t **mq_open**(const char *name, int oflag, mode_t mode, struct mq_attr *attr);
mqd_t **mq_getattr**(mqd_t mqdes, struct mq_attr *attr);
mqd_t **mq_setattr**(mqd_t mqdes, struct mq_attr *newattr, struct mq_attr *oldattr);
int **mq_send**(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);
ssize_t **mq_receive**(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);
int **mq_close**(mqd_t mqdes);
int **mq_unlink**(const char *name);
int **mq_notify**(mqd_t mqdes, const struct sigevent *notification);
int **kill**(pid_t pid, int signo);
int **raise**(int signo);
int **sigaction**(int signo, const struct sigaction *act, struct sigaction *oact);
int **sigprocmask**(int how, const sigset_t *set, sigset_t *oset);
int **pthread_sigmask**(int how, const sigset_t *set, sigset_t *oset);
int **sigpending**(sigset_t *set);
int **sigemptyset**(sigset_t *set);
int **sigfillset**(sigset_t *set);
int **sigaddset**(sigset_t *set, int signo);
int **sigdelset**(sigset_t *set, int signo);
int **sigismember**(const sigset_t *set, int signo);
int **pause**(void);
int **sigsuspend**(const sigset_t *sigmask);
int **sigwait**(const sigset_t *set, int *sig);
ssize_t **read**(int filedes, void *buf, size_t nbytes);
ssize_t **write**(int fildes, const void *buf, size_t nbyte);
int **close**(int filedes);
int **fcntl**(int fildes, int cmd, ...);
off_t **lseek**(int filedes, off_t offset, int whence);
int **ioctl**(int fildes, int request, ... /* arg */);
int **socket**(int *domain*, int *type*, int *protocol*);
int **bind**(int *sockfd*, const struct sockaddr **my_addr*, socklen_t *addrlen*);

uint16_t **htons**(uint16_t hvalue);
uint32_t **htonl**(uint32_t hvalue);
uint16_t **ntohs**(uint16_t hvalue);
uint32_t **ntohl**(uint32_t hvalue);
int **sendto**(int sockfd, const void *buff, size_t nbytes, int flags, const struct sockaddr *to, socklen_t tolen);
int select(int maxfdp1, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
int **recvfrom**(int sockfd, void *buff, size_t nbytes, int flags, struct sockaddr *from, socklen_t *fromlen);
in_addr_t **inet_addr**(const char *dotted);
char ***inet_ntoa**(struct in_addr network);
int **listen**(int sockfd, int backlog);
int **connect**(int sockfd, const struct sockaddr *serv_addr,socklen_t addrlen);
int **accept**(int sockfd, struct sockaddr *addr,socklen_t *addrlen);
int **close**(int sockfd);
int **shutdown**(int sockfd, int howto);
FD_ZERO (fd_set *set);
FD_SET (int fd, fd_set *set);
FD_CLR (int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
int **getsockname**(int socket, struct sockaddr *restrict address, socklen_t *restrict address_len);